3LAY

Receiving results from 3lay: webhook guide

For you and your AI coding agent. You can paste this whole page into your agent (e.g. Replit Agent) and ask it to build the receiver. It contains everything the agent needs.

In one paragraph

When 3lay finishes processing an email sent to your 3lay address, it sends the results to **your webhook**: an HTTPS address on your server. 3lay makes a `POST` request with a JSON body, and you reply `200 OK`. Each request is **signed** with a secret only you and 3lay know, so your server can check that the request really came from 3lay and wasn't changed. 3lay follows the open Standard Webhooks specification, so a ready-made library does the checking for you.

What to build (instructions for the coding agent)

Build one HTTPS endpoint, e.g. `POST /webhooks/3lay`, that:

1. **Reads the raw request body** as a string or bytes, *before* any JSON parsing. The signature covers the exact bytes 3lay sent; parsing and re-serialising the JSON changes them, and verification then fails.
2. **Verifies the signature** with the official `standardwebhooks` library and the secret from an environment variable (`THREELAY_WEBHOOK_SECRET`). If verification fails, reply `401` and do nothing else.
3. **Ignores duplicates:** if this `webhook-id` header was already processed, reply `200` and stop. (3lay retries when it doesn't get a 2xx, so the same delivery can arrive more than once.)
4. **Stores the event, then replies `200` quickly**, within 10 seconds. Do slow work (emails, other API calls) after replying, or in a background job.
5. **Handles both event types:** `event.completed` and `event.failed` (below).

Never hard-code the secret in source code, and never log it or the full request body: the body contains personal data from your users' documents.

The request 3lay sends

```
POST <your webhook URL>
content-type: application/json
user-agent: 3lay-webhooks/1
webhook-id: 7f3c2a10-5b8e-4c1d-9a2f-0e6b4d8c1a23-r1
webhook-timestamp: 1791969657
webhook-signature: v1,K5oZfzN95Z9UVu1EsfQmfVNQhnkZ2pj9o9NDN/H/pI4=
```

Header	Meaning
`webhook-id`	Unique ID of this delivery: the same on every retry of it. Use it to ignore duplicates.

Header	Meaning
`webhook-timestamp`	When this attempt was sent (Unix seconds). The library rejects requests older than 5 minutes, which stops replays.
`webhook-signature`	`v1,<base64 HMAC-SHA256>` of `<webhook-id>.<webhook-timestamp>.<raw body>`, keyed with your secret. Sometimes several, separated by spaces (during a secret rotation); the library accepts any that matches.

The body: an "envelope"

Every delivery has the same shape. Only `items[].data` is specific to you: it contains the fields you set up with Boardy, 3lay's setup assistant.

Example `event.completed` (invented data):


```
{
  "envelope_version": 1,
  "event_type": "event.completed",
  "event_id": "7f3c2a10-5b8e-4c1d-9a2f-0e6b4d8c1a23",
  "run": 1,
  "case_ref": "7f3c2a10-5b8e-4c1d-9a2f-0e6b4d8c1a23",
  "received_at": "2026-10-14T09:20:57.731Z",
  "finished_at": "2026-10-14T09:21:31.402Z",
  "sender": "jane.actor@example.com",
  "status": "completed",
  "summary": { "accepted": 1, "needs_review": 1, "rejected": 0 },
  "config_version": 2,
  "dashboard_url": "https://app.3lay.live/events/7f3c2a10-5b8e-4c1d-9a2f-0e6b4d8c1a23",
  "items": [
    {
      "source": { "attachment": "statement-october.pdf", "location": "0", "forwarded_from": null },
      "document_type": "income_statement",
      "schema": { "id": "rolepay-income-statement", "version": 1 },
      "verdict": "accepted",
      "confidence": null,
      "reasons": [],
      "data": {
        "payment_date": "31/10/2026",
        "total_net": 1240.5,
        "lines": [
          {
            "description": "Commercial shoot",
            "gross": 1500.0,
            "net": 1240.5,
            "deductions": [{ "category": "agency_commission", "amount": 259.5 }]
          }
        ]
      },
```

```
      "error": null
    },
    {
      "source": { "attachment": "photo.jpg", "location": "1", "forwarded_from": null },
      "document_type": "income_statement",
      "schema": { "id": "rolepay-income-statement", "version": 1 },
      "verdict": "needs_review",
      "confidence": null,
      "reasons": [
        "line_totals_add_up[0]: Line totals don't add up: 800.00 (gross) - 160.00
(deductions[].amount) = 640.00, but net is 600.00"
      ],
      "data": {
        "payment_date": "15/10/2026",
        "total_net": 600.0,
        "lines": [
          {
            "description": "Voice-over",
            "gross": 800.0,
            "net": 600.0,
            "deductions": [{ "category": "agency_commission", "amount": 160.0 }]
          }
        ]
      },
      "error": null
    }
  ],
  "analyses": [],
  "error": null
}
```


Example `event.failed`: nothing in the email could be processed.


```
{
  "envelope_version": 1,
  "event_type": "event.failed",
  "event_id": "7f3c2a10-5b8e-4c1d-9a2f-0e6b4d8c1a23",
  "run": 1,
  "case_ref": "7f3c2a10-5b8e-4c1d-9a2f-0e6b4d8c1a23",
  "received_at": "2026-10-14T09:20:57.731Z",
  "finished_at": "2026-10-14T09:21:31.402Z",
  "sender": "jane.actor@example.com",
  "status": "failed",
  "summary": { "accepted": 0, "needs_review": 0, "rejected": 0 },
  "config_version": 2,
  "dashboard_url": "https://app.3lay.live/events/7f3c2a10-5b8e-4c1d-9a2f-0e6b4d8c1a23",
  "items": [],
```

```
  "analyses": [],
  "error": { "code": "no_documents_found", "message": "The email has no attachments." }
}
```

Field reference

Field	Meaning
`event_type`	`event.completed` : the email was processed (each document has its own verdict). `event.failed` : nothing usable came out of it; see `error` .
`event_id`	One email = one event.
`run`	`1` normally. If 3lay reprocesses an email, you get a new delivery with `run` 2, 3… and a new `webhook-id` : treat it as the newer result for the same `event_id` .
`case_ref`	Groups related events; today it equals `event_id` .
`sender`	The email address the email came from (your end user), or `null` .
`received_at` , `finished_at`	When 3lay received the email and finished it (ISO 8601, UTC).
`summary`	How many documents got each verdict.
`config_version`	Which version of your setup processed it.
`dashboard_url`	A link to the event in 3lay's dashboard. *The event page isn't available yet: don't rely on this link.*
`items`	One per document (attachment) in the email.
`items[].source`	`attachment` : the file name; `location` : its position in the email (e.g. `"1 > 0"` inside a forwarded email); `forwarded_from` : the original sender if it was forwarded.
`items[].verdict`	`accepted` : extracted and every check passed. `needs_review` : extracted, but a check failed (see `reasons`); worth a human look. `rejected` : couldn't be processed; `data` is `null` and `error` says why.
`items[].reasons`	Why the verdict isn't `accepted` , in plain words. Empty when accepted.
`items[].data`	The extracted fields: the ones you set up with Boardy. A value that isn't on the document is `null` . Dates are copied as printed.
`items[].confidence`	Always `null` for now (reserved).
`analyses`	Always `[]` for now (reserved).
`error`	On `event.failed` (and on rejected items): `{code, message}` . Codes: `no_documents_found` , `unsupported_file_type` , `document_unreadable` , `processing_error` , `processing_timeout` .

New fields may be added to the envelope over time, so ignore fields you don't recognise rather than rejecting the request.

Replying, and retries

Your reply	What 3lay does
Any `2xx` within 10 seconds	Delivered. Done.
`5xx`, `429`, a timeout (over 10 s) or no connection	Retries 3 more times, after about 15, 30 and 60 seconds (a `429` with `Retry-After` is honoured, up to 60 s). Each retry has the same `webhook-id`, a new timestamp and a new signature.
A redirect (`3xx`)	Gives up: redirects aren't followed. Use the final URL.
Any other `4xx` (e.g. `400`, `401`, `404`)	Gives up: your server refused it.

So reply `2xx` once you've safely stored the event, `401` for a bad signature, and `5xx` only when you want 3lay to try again later.

Code: Node.js (Express)

```
npm install express standardwebhooks
```

```
import express from "express";
import { Webhook } from "standardwebhooks";

const app = express();
const webhook = new Webhook(process.env.THREELAY_WEBHOOK_SECRET); // "whsec_..."
const seen = new Set(); // use your database in production

// express.text keeps the raw body: don't use express.json() on this route.
app.post("/webhooks/3lay", express.text({ type: "application/json" }), (req, res) => {
  let envelope;
  try {
    envelope = webhook.verify(req.body, req.headers); // checks signature and timestamp
  } catch {
    return res.status(401).send("invalid signature");
  }

  const deliveryId = req.headers["webhook-id"];
  if (seen.has(deliveryId)) return res.sendStatus(200); // a retry we already have
  seen.add(deliveryId);

  // Store it first, reply, then do anything slow.
  if (envelope.event_type === "event.completed") {
    for (const item of envelope.items) {
      // item.verdict: "accepted" | "needs_review" | "rejected"
      // item.data: your fields (null when rejected)
```

```
    }
  } else {
    // event.failed: envelope.error.code / envelope.error.message
  }
  res.sendStatus(200);
});

app.listen(process.env.PORT || 3000);
```

Code: Python (Flask)

```
pip install flask standardwebhooks
```

```
import os

from flask import Flask, request
from standardwebhooks.webhooks import Webhook, WebhookVerificationError

app = Flask(__name__)
webhook = Webhook(os.environ["THREELAY_WEBHOOK_SECRET"])  # "whsec_..."
seen = set()  # use your database in production


@app.post("/webhooks/3lay")
def receive_3lay():
    try:
        # The raw body, exactly as sent: not request.json.
        envelope = webhook.verify(request.get_data(), dict(request.headers))
    except WebhookVerificationError:
        return "invalid signature", 401

    delivery_id = request.headers["webhook-id"]
    if delivery_id in seen:
        return "", 200  # a retry we already have
    seen.add(delivery_id)

    if envelope["event_type"] == "event.completed":
        for item in envelope["items"]:
            ...  # item["verdict"], item["data"]
    else:
        ...  # envelope["error"]["code"], envelope["error"]["message"]
    return "", 200
```

Setting it up, step by step

1. **Build and publish the receiver.** Ask your agent to build it from this page. Publish it with a **Replit Deployment**, so it has a stable public `https://` address that's always on. The address of the development preview changes and sleeps, so 3lay can't rely on it.
2. **Give the address to Boardy** in 3lay (Dashboard → *Set up processing*), e.g. `https://your-app.replit.app/webhooks/3lay`, then approve your setup.
3. **Copy the signing secret** shown once after approving (it starts with `whsec_`). Save it in Replit as a **Secret** named `THREELAY_WEBHOOK_SECRET`, then redeploy so the app picks it up.
4. **Test it:** email a document to your 3lay address (`<username>@in.3lay.live`) and watch your app receive it. Use an invented or anonymised document while testing.

Lost the secret, or think it leaked? Dashboard → *Webhooks* → *Rotate secret*. You get a new secret, shown once. The old one keeps working for 24 hours, with both signatures sent meanwhile, so update the Replit Secret and redeploy within that time.

Changing the URL or which events you receive: ask Boardy, then approve the updated setup. It takes effect from the next email.

Checklist

- [ ] HTTPS endpoint that accepts `POST` with a JSON body
- [ ] Verifies the signature on the **raw** body with `standardwebhooks` and `THREELAY_WEBHOOK_SECRET`; `401` if invalid
- [ ] Ignores repeated `webhook-id`s
- [ ] Stores the event and replies `2xx` within 10 seconds
- [ ] Handles `event.completed` (each item's `verdict`, `data`, `reasons`) and `event.failed` (`error`)
- [ ] Ignores unknown fields
- [ ] Secret only in Replit Secrets; never logs the secret or full bodies